

Thanks for providing the full codebase. With the raw code and the timing telemetry you provided earlier, I've pinpointed the exact causes of the createMasterList crash and the extreme slowdowns in the Format Raw Data workflow.

Here are the critical fixes you need to apply to stabilize the framework.

1. Fix the createMasterList Crash (getLastRow of undefined)

The crash happens because of a property name mismatch in your dashboard config object. When creating the configuration, the index is stored as `sectionIndex`, but the trimmer function attempts to read `dashboardIndex`. Because it reads undefined, it passes null to the row reader, which eventually tries to run `.getLastRow()` on nothing.

Update trimOutputSheetToDataSize_: Find this function (around line 1350) and update the globals variable declaration to use `sectionIndex`:

JavaScript

```
function trimOutputSheetToDataSize_(sheet, rowCount, sheetType) {
  if (!sheet) return;
  const dashboard = loadDashboardConfig_();
  const sheetDef = getSheetDefinitionByType_(dashboard, sheetType);
  // FIX: Change dashboard.dashboardIndex to dashboard.sectionIndex
  const globals = loadGlobalSettings_(dashboard && dashboard.sectionIndex ? dashboard.sectionIndex : null);
  const templateRows = resolveTemplateRowCount_(sheetDef || { templateRowMode: "FIXED" }, globals || {});
  const desiredRows = Math.max(DATA_START_ROW + Number(rowCount || 0) + 50, templateRows || DATA_START_ROW);
  forceSheetRowCount_(sheet, desiredRows);
}
```

Update loadGlobalSettings_: Find this function (around line 1850) and update its fallback to match:

JavaScript

```
function loadGlobalSettings_(sheet) {
  // FIX: Change .dashboardIndex to .sectionIndex
  sheet = sheet || loadDashboardConfig_().sectionIndex;
  const rows = readDashboardSectionRows_(sheet, RFF_SECTION_GLOBAL);
  // ... rest of function remains unchanged
}
```

2. Fix the 98-Second Banner Sync (The Hidden Loop Tax)

The 98-second bottleneck in `syncRawDataBannerColumns_` is a classic Google Apps Script trap. Inside your loop, you are comparing values using `valuesAreEqual_`, which relies on `normalizeCompareValue_`.

`normalizeCompareValue_` and `normalizeKeyPart_` both call `Utilities.formatDate()`. While `Utilities` looks like native Javascript, it is actually a Google Java API wrapper. Calling it inside a 6,500-row loop crosses the sandbox boundary tens of thousands of times, adding ~2ms of latency per call. This causes a massive pileup.

Update both normalizers to use native Javascript Date methods:

JavaScript

```
function normalizeCompareValue_(value) {
  if (value instanceof Date && !isNaN(value.getTime())) {
    // FIX: Use native JS to avoid slow Utilities API boundary crossing
    const y = value.getFullYear();
    const m = String(value.getMonth() + 1).padStart(2, '0');
    const d = String(value.getDate()).padStart(2, '0');
    return `${y}-${m}-${d}`;
  }
  if (value === null || value === undefined) return "";
  const text = String(value).trim();
  if (text === "") return "";
  if (text.length > 5 && /\d{1,2}[\V\.\-]\d{1,2}[\V\.\-]\d{2,4}/.test(text)) {
    const parsed = coerceToValidDate_(text);
    if (parsed) {
      const y = parsed.getFullYear();
      const m = String(parsed.getMonth() + 1).padStart(2, '0');
      const d = String(parsed.getDate()).padStart(2, '0');
      return `${y}-${m}-${d}`;
    }
  }
  return text.toLowerCase().replace(/s+/g, " ");
}

function normalizeKeyPart_(value) {
  if (value === null || value === undefined) return "";
  if (value instanceof Date && !isNaN(value.getTime())) {
    // FIX: Use native JS to avoid slow Utilities API boundary crossing
    const y = value.getFullYear();
    const m = String(value.getMonth() + 1).padStart(2, '0');
    const d = String(value.getDate()).padStart(2, '0');
    return `${y}${m}${d}`;
  }
  return String(value).trim().toLowerCase().replace(/s+/g, " ");
}
```

3. Fix the 240-Second Format Paste (Redundant API Calls)

In `createRawDataOutputSheetFromTemplateFast_`, the script duplicates the template using `template.copyTo(ss)`. Google natively preserves frozen rows, frozen columns, column widths, and tab colors during this duplication.

However, immediately following the copy, your code runs a loop to query the template and overwrite the new sheet's column widths cell-by-cell (around line 2580): `for (let col = 1; col <= width; col++) { sheet.setColumnWidth(col, template.getColumnWidth(col)); }`

For a 54-column sheet, this makes 108 redundant, slow grid calls.

Remove the redundant UI grid loops in createRawDataOutputSheetFromTemplateFast_: Simply delete this entire try...catch block from the function. The sheet already has these properties from the .copyTo() command.

JavaScript

```
// REMOVE THIS ENTIRE BLOCK to save significant runtime:
/* try {
for (let col = 1; col <= width; col++) {
sheet.setColumnWidth(col, template.getColumnWidth(col));
}
sheet.setFrozenRows(template.getFrozenRows());
sheet.setFrozenColumns(template.getFrozenColumns());
if (template.getTabColor()) sheet.setTabColor(template.getTabColor());
} catch (err) {
Logger.log("Raw Data template sheet settings copy skipped for " + outputName + ": " + err.message);
}
*/
```

Applying these three fixes will resolve the Master List failure and likely shave minutes off your Raw Data formatter workflow.

Would you like to review the batching strategy for the Dashboard Quality sections next, so we can get that workflow's runtime under its 30-second benchmark?